

```

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Image size
img_size = 128
batch_size = 32

# Data generators
train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_data = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical'
)

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

test_data = test_datagen.flow_from_directory(
    'dataset/test',
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical'
)

# CNN Model
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax') # 3 classes
])

# Compile
model.compile(optimizer='adam',
              loss='categorical_crossentropy',

```

```

        metrics=['accuracy'])

# Train
model.fit(train_data, epochs=5, validation_data=test_data)

# Evaluate
loss, acc = model.evaluate(test_data)
print("Accuracy:", acc)
# =====
# 1. IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans

# =====
# 2. LOAD DATASET
# =====
df = pd.read_csv("data.csv") # change file name

print("\nFirst 5 rows:\n", df.head())
print("\nInfo:\n")
print(df.info())

# =====
# 3. DESCRIPTIVE STATISTICS (EDA)
# =====
print("\nDescriptive Statistics:\n", df.describe())

# =====
# 4. DATA PREPROCESSING (AUTO)
# =====

# Separate numerical & categorical
num_cols = df.select_dtypes(include=np.number)
cat_cols = df.select_dtypes(exclude=np.number)

# Handle missing values

```

```

num_cols = num_cols.fillna(num_cols.mean())
cat_cols = cat_cols.fillna("Unknown")

# Encode categorical
cat_cols = pd.get_dummies(cat_cols, drop_first=True)

# Combine
df = pd.concat([num_cols, cat_cols], axis=1)

# =====
# 5. SCALING
# =====
scaler = StandardScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

# =====
# 6. VISUALIZATION (5 TYPES)
# =====

# Histogram
df.hist(figsize=(10,8))
plt.show()

# Boxplot
sns.boxplot(data=df)
plt.show()

# Scatter plot
if df.shape[1] >= 2:
    sns.scatterplot(x=df.iloc[:,0], y=df.iloc[:,1])
    plt.show()

# Heatmap
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()

# Pairplot
sns.pairplot(df)
plt.show()

# =====
# 7. CORRELATION & COVARIANCE
# =====
print("\nCorrelation:\n", df.corr())
print("\nCovariance:\n", df.cov())

# =====
# 8. MODEL SELECTION (AUTO DETECT)

```

```

# =====

# Assume last column is target (you can change)
target = df.columns[-1]

X = df.drop(target, axis=1)
y = df[target]

# Check if classification or regression
if y.nunique() <= 10:
    print("\nDetected: CLASSIFICATION")

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    models = {
        "Logistic": LogisticRegression(max_iter=1000),
        "Decision Tree": DecisionTreeClassifier(),
        "Random Forest": RandomForestClassifier()
    }

    for name, model in models.items():
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        print(name, "Accuracy:", accuracy_score(y_test, pred))

else:
    print("\nDetected: REGRESSION")

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    model = LinearRegression()
    model.fit(X_train, y_train)

    pred = model.predict(X_test)
    print("MSE:", mean_squared_error(y_test, pred))

# =====
# 9. CLUSTERING
# =====

kmeans = KMeans(n_clusters=3)
clusters = kmeans.fit_predict(X)

plt.scatter(X.iloc[:,0], X.iloc[:,1], c=clusters)
plt.title("Clustering")
plt.show()

```